

Ans. to the q. no. - 01.

a) For the above scenario, Waterfall model won't be valid as for the above scenario the requirements can be change over time. Waterfall model says, after finishing the first phase, in the second phase can be move. which doesn't go with the above scenario.

b) Iterative and Incremental model would be suitable for the above the scenarios as these models goes with constant requirement changing, as well as offers flexibility of projects.

Ans. to the q. no. - 02

a) For the above scenario Singleton pattern should be implemented. Singleton Pattern only allows one instance which allows global point access. So, to manage the entire departments printing queue, one single instance server should be implemented

b)

```
class print_queue:
```

```
    instance = None
```

```
    def new(cls):
```

```
        if cls.instance == None:
```

```
            cls.instance = (print_queue, cls.new(cls))
```

```
    an_instance.
```

```
    cls.instance.print_queue = [ ]
```

```
    return cls.instance
```



```
def add_queue(self, print_queue):  
    self.print_queue.append(print_queue)
```

```
def select_printer(self):  
    return 'p1'
```

```
print = printQueue()
```

```
print.add_queue('pdf1')
```

```
print.add_queue('pdf2')
```

```
selected = print.select_printer()
```

```
print('selecte: {p}')
```


Ans. to the q. no. - 03

a) Hence,

$$NMO = 0$$

$$NMI = 2$$

$$NMA = 1$$

$$DIT = 1$$

Events



Sports.

Now,

$$SIx = \frac{NMO \times DIT}{NMA + NMI + NMO}$$

$$= \frac{0 \times 1}{1 + 2 + 1}$$

$$= \frac{0}{4}$$

$$= 0$$

Ans. to the q. no. 104

a) 3 code smell:

1. Inappropriate naming:

in 'Customer' class there used `getNM` and in 'Item' class there used `getPr` which are cleared name. There could be like `getName()` and `getPrice()`.

2. Duplicate code:

Discount method is repeated in class 'ShoppingSystem' and 'Customer' where it can be extracted into its own method within the 'Customer' class.

Like:

```
public Customer (String name, boolean isMember) {  
    this.name = name; this.isMember = isMember;  
}
```


3. Undefined Variables:

→ In 'shoppingSystem' the 'addItem' hasn't any variable type.

Like: shoppingCart.addItem(string(mmouse));

b)

Refactoring: -

1. Incomplete class Definitions

```
class Item {  
    private String name;  
    private double price;  
  
    public Item(String name, double price)  
  
        this.name = name;  
        this.price = price;  
  
}  
  
    public double getPrice() {  
        return price;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```


2. Unused variable;

private static final int

count variable = 0

3. Magic number;

4. Variable Naming -

Item - name = 'laptop'

Item - price = '1200'

Improving name

Like

Item laptop = new Item('laptop', 1200);